

# Otimização de Joins Desbalanceados no Apache Spark: Do Pandas ao Big Data

## 1. O Problema: Por que o Spark é diferente do Pandas?

No **Pandas**, os dados estão inteiros na memória RAM de um único computador. Quando se faz um `merge` (join), o processador acessa as duas tabelas lado a lado. Se demorar, é apenas uma questão de velocidade da CPU.

No **Spark**, os dados estão fatiados (particionados) e espalhados por vários computadores (executors). Para juntar (fazer join) o Dataframe A com o Dataframe B, o Spark precisa garantir que as linhas com a mesma chave (ex: `id_usuario`) estejam **no mesmo computador**.

- **O "Gargalo" (Shuffle):** Se os dados não estiverem alinhados, o Spark precisa mover gigabytes de dados através da rede de cabos entre os computadores. Isso se chama **Shuffle**. É lento, custoso e propenso a falhas.

## 2. O Cenário do Desafio

Imagine um join entre:

1. **Tabela Fato (Big):** 100 milhões de registros (ex: Transações de Vendas).
2. **Tabela Dimensão (Small):** 1 mil registros (ex: Cadastro de Lojas).

Além da diferença de tamanho, temos o problema de **Data Skew** (distorção): Algumas chaves aparecem muito mais que outras (ex: uma loja "Mega Store" tem 5 milhões de vendas, enquanto as outras têm poucas).

## 3. Estratégias de Otimização

Aqui estão as três abordagens para resolver esse problema, traduzidas para a lógica de quem conhece Pandas:

### A. Broadcast Join (O "Copy-Paste")

No Pandas, isso é o padrão implícito. No Spark, forçamos essa estratégia.

- **Como funciona:** Em vez de mover a tabela gigante (100M linhas) pela rede para encontrar a pequena, o Spark envia uma cópia completa da tabela pequena (1000 linhas) para **todos** os computadores que contêm pedaços da tabela grande.
- **Analogia:** Em vez de fazer 10 milhões de alunos (Tabela Grande) irem até a secretaria ver a lista de notas (Tabela Pequena), você cola uma cópia da lista de notas na porta de cada sala de aula. Ninguém precisa sair do lugar (zero Shuffle da tabela grande).
- **Quando usar:** Quando um dos lados do join é pequeno o suficiente para caber na memória (< 8GB, mas idealmente < 100MB para ser rápido).

### B. Reparticionamento (A "Reorganização")

- **Como funciona:** Antes do join, redistribuímos os dados explicitamente para garantir que partições tenham tamanhos similares ou que as chaves estejam melhor organizadas.
- **No Pandas:** Seria como dar um `sort_values` ou garantir um índice limpo antes de um merge complexo.
- **No Spark:** Aumentamos ou diminuimos o número de partições para aumentar o paralelismo. Isso custa um shuffle inicial, mas pode acelerar o processamento subsequente se os dados estiverem muito fragmentados.

### C. Salting (O "Truque do Sal")

Esta é a técnica mais avançada para resolver o **Data Skew** (quando uma única chave trava um executor).

- **O Problema:** Se a "Loja A" tem 5 milhões de vendas, o computador responsável pela chave "Loja A" vai trabalhar por horas enquanto os outros terminam em segundos e ficam ociosos.
- **A Solução (Salting):** Nós "temperamos" as chaves com números aleatórios para enganar o Spark e forçá-lo a dividir o trabalho.
  1. Na tabela grande, transformamos a chave `Loja A` em `Loja A_1`, `Loja A_2`, `Loja A_3`... (aleatoriamente).
  2. Na tabela pequena, nós **explicamos** a linha da `Loja A` para criar `Loja A_1`, `Loja A_2`, etc.
  3. Agora, o Spark acha que são chaves diferentes e envia para computadores diferentes.
- **Resultado:** O trabalho da "Loja A" é dividido entre vários computadores.

## 4. Proposta do Exercício Prático (Google Colab)

O objetivo é simular, medir e comparar o tempo de execução e o tráfego de dados (Shuffle Write/Read) nas seguintes etapas:

1. **Setup:**
  - Gerar Dataframe `df_large` (100M linhas) com skew intencional (ex: 80% dos dados em apenas 2 chaves).

- Gerar Dataframe `df_small` (1 mil linhas).

## 2. Teste 1: Join Padrão (SortMergeJoin)

- Deixar o Spark decidir. Observar a lentidão devido ao Skew (um executor trabalhando muito mais que os outros).

## 3. Teste 2: Broadcast Hash Join

- Forçar o envio do `df_small` para os nós.
- *Expectativa*: Execução quase instantânea, pois elimina o shuffle do `df_large`.

## 4. Teste 3: Salting (Caso Avançado)

- Implementar a lógica de sufixos aleatórios.
- *Expectativa*: Melhoria significativa caso o Broadcast não fosse possível (simulando que a tabela pequena também fosse grande demais para broadcast).

## ✓ Packages e session spark

```
# 1. Instalar o PySpark (necessário no Google Colab)
#!pip install pyspark -q

# 2. Importar bibliotecas
import time
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lit, rand, floor, when, broadcast, concat, explode

# 3. Iniciar a Sessão Spark
# 'local[*]' usa todos os núcleos do processador disponíveis no Colab
# Config 'spark.driver.memory' tenta alocar 8GB de RAM para o Spark não travar
spark = SparkSession.builder \
    .appName("Desafio Join Skew") \
    .master("local[*]") \
    .config("spark.driver.memory", "8g") \
    .config("spark.sql.adaptive.enabled", "false") \
    .getOrCreate()

print("Spark configurado com sucesso!")
```

Spark configurado com sucesso!

## ✓ Gerar dataframes

```
# --- CONFIGURAÇÃO ---
NUM_ROWS = 100_000_000 # 100 Milhões (Se travar, reduza para 10_000_000)
NUM_KEYS = 1_000       # A tabela pequena terá 1000 registros

print(f"Gerando dataset com {NUM_ROWS/1_000_000} milhões de linhas...")

# 1. Gerar Tabela GRANDE (Fato) com SKEW (Distorção)
# Lógica: Se um número aleatório for menor que 0.95, a chave é '1'.
# Caso contrário, é um número aleatório entre 2 e 1000.
# Isso simula aquele cliente ou loja gigante que domina os dados.

df_large = spark.range(0, NUM_ROWS).select(
    when(rand() < 0.95, 1)                # 95% de chance de ser chave 1
    .otherwise(floor(rand() * NUM_KEYS) + 1) # 5% de chance de ser o resto
    .alias("join_key"),
    rand().alias("valor_transacao")        # Uma coluna qualquer de valor
)

# 2. Gerar Tabela PEQUENA (Dimensão)
# Apenas IDs de 1 a 1000 e um nome fictício
```

```
df_small = spark.range(1, NUM_KEYS + 1).select(
    col("id").alias("join_key"),
    lit("Dados da Loja").alias("descricao_loja")
)

# Forçar o Spark a materializar os dados (cache) para o tempo de criação não afetar o
print("Cacheando os dataframes na memória...")
df_large.cache()
df_small.cache()
print(f"Contagem final Tabela Grande: {df_large.count()} (Isso força o processamento i
print("Dados prontos!")
```

```
Gerando dataset com 100.0 milhões de linhas...
Cacheando os dataframes na memória...
Contagem final Tabela Grande: 100000000 (Isso força o processamento inicial)
Dados prontos!
```

#### ✓ Função para realizar o benchmark dos joins

```
def benchmark_join(nome_teste, df_resultado):
    start_time = time.time()

    # .write.format("noop") é um truque para executar toda a lógica do join
    # sem perder tempo salvando em disco. É join puro.
    df_resultado.write.format("noop").mode("overwrite").save()

    end_time = time.time()
    print(f"--- {nome_teste} ---")
    print(f"Tempo de execução: {end_time - start_time:.2f} segundos")
    print("-" * 30)
```

#### ✓ CENÁRIO 1: Join Padrão (SortMergeJoin) - O Lento

- Desabilitamos o broadcast automático para simular que o Spark escolheu o jeito "burro"

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)

print("Iniciando Join Padrão (pode demorar um pouco devido ao Skew)...")
df_join_standard = df_large.join(df_small, on="join_key", how="inner")

benchmark_join("1. JOIN PADRÃO (Com Skew)", df_join_standard)
```

```
Iniciando Join Padrão (pode demorar um pouco devido ao Skew)...
--- 1. JOIN PADRÃO (Com Skew) ---
Tempo de execução: 147.32 segundos
-----
```

#### ✓ CENÁRIO 2: Broadcast Join - A Bala de Prata (para tabelas pequenas)

- Aqui forçamos explicitamente o envio da tabela pequena para a memória de todos os nós

```
print("Iniciando Broadcast Join...")
df_join_broadcast = df_large.join(broadcast(df_small), on="join_key", how="inner")

benchmark_join("2. BROADCAST JOIN", df_join_broadcast)
```

```
Iniciando Broadcast Join...
--- 2. BROADCAST JOIN ---
Tempo de execução: 12.49 segundos
-----
```

#### ✓ CENÁRIO 3: Salting - A Solução "Hacker" (Quando Broadcast não cabe)

- Imagine que a tabela pequena também fosse grande demais para broadcast.
- Teríamos que usar Salting para distribuir a chave "1".

```
print("Preparando dados para Salting...")
SALT_NUMBER = 20 # Vamos dividir a chave "1" em 20 pedaços

# A. Adicionar Sal na Tabela GRANDE (Sufixo aleatório de 0 a 19)
df_large_salted = df_large.withColumn(
    "salted_key",
    concat(col("join_key"), lit("_"), floor(rand() * SALT_NUMBER))
)

# B. Explodir a Tabela PEQUENA (Replicar cada linha 20 vezes, de 0 a 19)
# Isso faz a tabela pequena crescer 20x, mas permite o pareamento.
df_small_salted = df_small.withColumn(
    "salt_array", sequence(lit(0), lit(SALT_NUMBER - 1)) # Cria array [0, 1, ... 19]
).withColumn(
    "salt_exploded", explode(col("salt_array"))           # Explode: 1 linha vira 20
).withColumn(
    "salted_key",
    concat(col("join_key"), lit("_"), col("salt_exploded"))
).drop("salt_array", "salt_exploded")

# C. Fazer o Join pela chave "Salgada"
df_join_salted = df_large_salted.join(df_small_salted, on="salted_key", how="inner")

benchmark_join("3. SALTING JOIN", df_join_salted)
```

```
Preparando dados para Salting...
--- 3. SALTING JOIN ---
Tempo de execução: 119.82 segundos
-----
```

Esses resultados ilustram perfeitamente a teoria na prática. Vamos analisar o "porquê" de cada número, pois é aqui que você ganha a intuição de Engenheiro de Dados.

## ✓ 1. O Vencedor Indiscutível: Broadcast Join (12.49s)

- **Redução de Tempo:** ~91% mais rápido que o padrão.
- **O que aconteceu:** Como a tabela pequena (1000 linhas) cabia na memória, o Spark enviou uma cópia dela para cada executor.
- **Por que foi tão rápido?** O tráfego de rede (Shuffle) da tabela de 100 Milhões de linhas foi **ZERO**. Os dados grandes ficaram parados onde estavam e o join aconteceu localmente.
- **Lição:** *Sempre* que uma das tabelas for pequena (menor que algumas centenas de MBs), use Broadcast. É imbatível.

## 2. O Perdedor: Join Padrão (147.32s)

- **O Gargalo:** O problema aqui não foi processar 100 milhões de linhas. O Spark faz isso rápido. O problema foi processar **95 milhões de linhas em um único núcleo de CPU**.
- **O que aconteceu:** O Spark tentou agrupar todos os registros com a chave "1" no mesmo lugar. Enquanto um "caixa de supermercado" (executor) tinha uma fila de 95 milhões de pessoas, os outros caixas estavam vazios.
- **Consequência:** O tempo total é igual ao tempo do executor mais lento (o *Straggler*).

## 3. O "Meio-Termo": Salting Join (119.82s)

- **Redução de Tempo:** ~19% mais rápido que o padrão.
- **Análise Crítica:** Você pode estar pensando: "*Só isso? Por que não foi tão rápido quanto o Broadcast?*"
  - **Motivo 1 (O Custo do Shuffle):** Ao contrário do Broadcast, o Salting **ainda obriga** o Spark a mover os 100 Milhões de registros pela rede. Ele move de forma balanceada, mas mover dados gasta tempo.
  - **Motivo 2 (O Custo do Cálculo):** O Salting adicionou passos extras: gerar números aleatórios para 100 milhões de linhas, criar novas colunas de texto (concat) e "explodir" a tabela pequena (multiplicando-a por 20). Isso tem um custo computacional (CPU).

- **Quando o Salting Brilha?** O Salting é a solução para quando a tabela "pequena" **NÃO** cabe na memória.
  - Se sua tabela menor tivesse 50 Milhões de linhas (impossível de fazer Broadcast), o **Join Padrão** traria ou levaria horas, e o **Salting** seria a única forma de fazer o job rodar.

---

## Resumo da Performance

Imagine que você tem que transportar 1000 tijolos (Dados) usando caminhões (Executors).

1. **Join Padrão (Skew):** Você colocou 950 tijolos em um único caminhão e 50 tijolos distribuídos em outros 10 caminhões. O comboio teve que andar na velocidade do caminhão superlotado (147s).
  2. **Salting:** Você teve o trabalho extra de separar os tijolos manualmente antes de carregar (custo de CPU), mas garantiu que cada caminhão levasse exatamente 90 tijolos. O comboio andou numa velocidade média boa (119s).
  3. **Broadcast:** Você percebeu que o destino dos tijolos era logo ali. Em vez de mover os tijolos (Tabela Grande), você trouxe o pedreiro (Tabela Pequena) até a pilha de tijolos. Trabalho concluído quase instantaneamente (12s).
- 

## Exercício 2: alterando o código para simular um cenário onde o Broadcast é proibido

- Simular falta de memória, o que nos forçaria a utilização do método Salting

```
import gc

# Lista provável de dataframes criados no exercício anterior
dfs_para_limpar = [
    'df_large', 'df_small',
    'df_join_standard', 'df_join_broadcast', 'df_join_salted',
    'df_large_salted', 'df_small_salted', 'df_small_exploded'
]

print("Iniciando limpeza...")

for nome_var in dfs_para_limpar:
    if nome_var in globals():
        # 1. Tira do Cache do Spark (se estiver cacheado)
        try:
            globals()[nome_var].unpersist()
        except:
            pass

        # 2. Deleta a variável do Python
        del globals()[nome_var]

# 3. Força a limpeza da memória RAM
gc.collect()

print("Variáveis deletadas e memória liberada!")
```

```
Iniciando limpeza...
Variáveis deletadas e memória liberada!
```

```
# --- DADOS ---
NUM_ROWS = 100_000_000 # 100 Milhões
NUM_KEYS = 1_000

print("1. Gerando DataFrame com Skew agressivo (95% na chave '1')...")
df_large = spark.range(0, NUM_ROWS).select(
    when(rand() < 0.95, 1).otherwise(floor(rand() * NUM_KEYS) + 1).alias("join_key"),
    rand().alias("valor")
).repartition(200) # Força dados espalhados para garantir shuffle
```

```
df_small = spark.range(1, NUM_KEYS + 1).select(
    col("id").alias("join_key"),
    lit("Loja X").alias("desc")
)

print("2. Cacheando dados para isolar o teste...")
df_large.write.format("noop").mode("overwrite").save() # Materializa sem ocupar RAM do
df_small.cache()
df_small.count()
print("Dados prontos.\n")
```

1. Gerando DataFrame com Skew agressivo (95% na chave '1')...  
 2. Cacheando dados para isolar o teste...  
 Dados prontos.

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
print(">>> MODO HARDCORE ATIVADO: Broadcast Desabilitado (-1) <<<\n")

def benchmark(nome, df):
    start = time.time()
    # Ação de contagem força o processamento do join
    qtd = df.write.format("noop").mode("overwrite").save()
    end = time.time()
    print(f"--- {nome} ---")
    print(f"Tempo: {end - start:.2f} segundos")
    print("-" * 30)
```

>>> MODO HARDCORE ATIVADO: Broadcast Desabilitado (-1) <<<

#### ✓ TESTE A: Join Padrão (SortMergeJoin) - Vai sofrer com o Skew

```
print("Iniciando Join Padrão (Prepare-se para esperar)...")
df_standard = df_large.join(df_small, on="join_key", how="inner")
benchmark("JOIN PADRÃO (SEM BROADCAST)", df_standard)
```

Iniciando Join Padrão (Prepare-se para esperar)...  
 --- JOIN PADRÃO (SEM BROADCAST) ---  
 Tempo: 309.94 segundos  
 -----

#### ✓ TESTE B: Salting Join - O Salvador da Pátria

```
print("Preparando Salting...")
SALT_NUMBER = 200 # Dividindo a carga em 50 pedaços

# 1. Salgar a Tabela Grande
df_large_salted = df_large.withColumn(
    "salted_key",
    concat(col("join_key"), lit("_"), floor(rand() * SALT_NUMBER))
)

# 2. Explodir a Tabela Pequena (Multiplica tamanho por 50, mas distribui carga)
df_small_exploded = df_small.withColumn(
    "salt_array", sequence(lit(0), lit(SALT_NUMBER - 1))
).withColumn(
    "salt_exploded", explode(col("salt_array"))
).withColumn(
    "salted_key",
    concat(col("join_key"), lit("_"), col("salt_exploded"))
).drop("salt_array", "salt_exploded")

# 3. Join
print("Iniciando Salting Join...")
```

```
df_salTED_join = df_large_salTED.join(df_small_exploded, on="salTED_key", how="inner")
benchmark("SALTING JOIN", df_salTED_join)
```

```
Preparando Salting...
Iniciando Salting Join...
--- SALTING JOIN ---
Tempo: 338.68 segundos
-----
```

## Lição: "Não Existe Almoço Grátis" na Engenharia de Dados

Ao realizar testes de otimização de Joins no Spark (Google Colab), me deparei com um resultado contra-intuitivo: a técnica avançada de **Salting** foi *mais lenta* que o Join Padrão, mesmo resolvendo o problema de assimetria dos dados (Skew).

Isso ilustra perfeitamente a regra de ouro da otimização: **A técnica certa no hardware errado vira gargalo.**

Aqui está a análise técnica do que aconteceu:

### 1. O "Imposto" da Complexidade (Overhead)

O Salting não é mágica; é código extra que exige processamento. Antes mesmo de iniciar o Join, obriguei o Spark a realizar tarefas pesadas:

- Gerar 100 milhões de números aleatórios.
- Concatenar strings em 100 milhões de linhas.
- **Explodir a tabela pequena:** Multipliquei os registros por 50, aumentando o volume de dados trafegados.

Tudo isso tem um custo computacional (CPU). Chamamos isso de **Overhead**. Eu paguei um "imposto" alto para preparar os dados.

### 2. O Gargalo do Paralelismo (A Falta de "Braço")

A premissa do Salting é dividir uma tarefa gigante e travada em 50 tarefas menores para serem executadas **simultaneamente**.

- **Em um Cluster Real (Produção):** Se eu tenho 50 CPUs disponíveis, as 50 tarefas rodam ao mesmo tempo. O tempo cai drasticamente.
- **No Meu Teste (Google Colab):** O ambiente é "Single Node", limitado a apenas 2 vCPUs.

**O Resultado:** Eu dividi o trabalho em 50 pedaços, mas só tinha 2 "operários" para processá-los. O Spark pegou os 2 primeiros pedaços e deixou os outros 48 na fila. No final, o trabalho foi feito sequencialmente de qualquer forma, **somado** ao tempo gasto com o overhead da preparação.

---

## Conclusão

Este exercício reforçou três pilares fundamentais para arquitetura de Big Data:

1. **Broadcast é Rei:** Se a tabela menor cabe na memória, use Broadcast. É imbatível (no meu teste: 12s vs 300s).
2. **Skew é Fatal:** O join padrão sem tratamento demorou 5x mais devido à concentração de dados em uma única partição.
3. **Custo x Benefício:** Técnicas de cluster distribuído (como Salting) exigem um cluster distribuído. Tentar forçar técnicas de Big Data em ambientes com poucos recursos pode piorar a performance. Otimização sem hardware compatível é apenas complexidade desnecessária.